

LAPORAN PRAKTIKUM
STRUKTUR DATA
“SORTING LANJUTAN”



Disusun oleh:

Ndaru Pradana Gatot Suseno
2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya Laporan Praktikum Struktur Data 8 ini dapat diselesaikan. Laporan ini disusun sebagai dokumentasi pelaksanaan praktikum sekaligus sarana untuk memahami metode pengurutan data lanjutan dalam bahasa Java.

Praktikum kedelapan membahas algoritma Shell Sort, Quick Sort, dan Merge Sort beserta visualisasinya melalui antarmuka Java Swing. Ketiga algoritma tersebut menunjukkan strategi pengurutan yang berbeda, yaitu penyisipan dengan jarak tertentu, pemisahan berdasarkan pivot, dan pembagian data secara rekursif sebelum digabungkan kembali. Pada laporan tugas, pengurutan diterapkan pada data lagu berdasarkan durasi menggunakan Quick Sort.

Penulis menyampaikan terima kasih kepada dosen pengampu, asisten praktikum, dan semua pihak yang telah memberikan arahan selama kegiatan berlangsung. Pembahasan laporan disusun berdasarkan source code pada package `pekan8_2411532007` dan hasil pengujian program secara langsung. Penulis menyadari bahwa laporan ini masih memiliki kekurangan sehingga kritik dan saran yang membangun sangat diharapkan sebagai bahan perbaikan.

Padang, 2026

Ndaru Pradana G.S

DAFTAR ISI**Contents**

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
Latar Belakang	4
Tujuan Praktikum.....	4
Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
Dasar Teori.....	5
Program MergeSort_2411532007	6
Program QuickSort_2411532007	8
Program ShellSort_2411532007	10
Program BubbleSortGui_2411532007	12
Program MergeSortGui_2411532007	15
BAB III KESIMPULAN.....	20
DAFTAR PUSTAKA	21
LAPORAN TUGAS	22
Tujuan Tugas.....	22
Class Lagu_2411532007.....	23
Class Sorting_2411532007	24
Hasil Eksekusi Laporan Tugas.....	26
Analisis Hasil	26
Kesimpulan Laporan Tugas	26
Saran.....	26

BAB I

PENDAHULUAN

Latar Belakang

Pengurutan data atau sorting merupakan proses menyusun elemen berdasarkan aturan tertentu, misalnya dari nilai terkecil ke terbesar atau sebaliknya. Data yang telah terurut lebih mudah ditelusuri, dibandingkan, dan diolah pada proses berikutnya. Oleh karena itu, pemilihan algoritma sorting menjadi bagian penting dalam perancangan program, terutama ketika ukuran data bertambah.

Praktikum ini mempelajari Shell Sort, Quick Sort, dan Merge Sort. Shell Sort meningkatkan proses insertion sort melalui penggunaan gap, Quick Sort mempartisi data berdasarkan pivot, sedangkan Merge Sort menerapkan pendekatan divide and conquer. Selain implementasi pada array bilangan, praktikum juga menggunakan Java Swing untuk menampilkan langkah pengurutan serta menerapkan Quick Sort pada objek lagu berdasarkan durasinya.

Tujuan Praktikum

1. Memahami konsep dan tahapan pengurutan menggunakan Shell Sort, Quick Sort, dan Merge Sort.
2. Menerapkan algoritma sorting pada array bilangan bulat dalam bahasa Java.
3. Memahami penggunaan rekursi, pivot, partisi, gap, dan proses merge.
4. Membuat visualisasi langkah Bubble Sort dan Merge Sort menggunakan Java Swing.
5. Menerapkan Quick Sort untuk mengurutkan objek lagu berdasarkan durasi.

Manfaat Praktikum

Praktikum ini memberikan pemahaman mengenai perbedaan strategi algoritma sorting, penggunaan rekursi dan partisi, serta cara memvisualisasikan perubahan data pada setiap langkah. Pengetahuan tersebut bermanfaat untuk memilih metode pengurutan yang sesuai dengan karakteristik data dan kebutuhan program.

BAB II

PEMBAHASAN

Dasar Teori

Sorting merupakan proses menata sekumpulan data berdasarkan kunci pengurutan. Dalam konteks array bilangan, kunci tersebut adalah nilai elemen, sedangkan pada data objek kunci dapat berupa salah satu atribut, seperti durasi lagu. Algoritma sorting dibedakan berdasarkan cara membandingkan, memindahkan, membagi, dan menggabungkan data.

Shell Sort bekerja dengan membandingkan elemen yang berjarak tertentu menggunakan nilai gap. Gap secara bertahap diperkecil hingga bernilai satu sehingga tahap terakhir menyerupai insertion sort pada data yang sebagian besar telah terurut.

Quick Sort menggunakan pendekatan divide and conquer. Data dipartisi berdasarkan pivot sehingga nilai yang lebih kecil berada di sebelah kiri dan nilai yang lebih besar berada di sebelah kanan. Pada program ini, pemilihan pivot dibantu metode median of three.

Merge Sort membagi array menjadi dua bagian secara rekursif hingga setiap bagian berisi satu elemen. Bagian-bagian tersebut kemudian digabungkan kembali dalam urutan yang benar. Kompleksitas waktu Merge Sort pada kondisi umum adalah $O(n \log n)$, tetapi membutuhkan array tambahan selama proses penggabungan.

Java Swing menyediakan komponen grafis seperti JFrame, JPanel, JLabel, JTextField, JButton, dan JTextArea. Komponen tersebut digunakan pada praktikum untuk memperlihatkan data array, langkah perbandingan, pertukaran, dan penggabungan secara visual.

Program MergeSort_2411532007

Class MergeSort_2411532007 mengimplementasikan Merge Sort pada array bilangan bulat. Method sort_2007 membagi rentang data menjadi dua bagian secara rekursif, sedangkan method Merge_2007 menggabungkan dua bagian yang telah terurut ke dalam array utama.

Source Code MergeSort_2411532007

```

1 package pekan8_2411532007;
2
3 public class MergeSort_2411532007 {
4     void Merge_2007(int arr_2007[], int l_2007, int m_2007, int r_2007) {
5         int n1_2007 = m_2007 - l_2007 + 1;
6         int n2_2007 = r_2007 - m_2007;
7
8         int L_2007[] = new int[n1_2007];
9         int R_2007[] = new int[n2_2007];
10
11         for (int i_2007 = 0; i_2007 < n1_2007; ++i_2007) {
12             L_2007[i_2007] = arr_2007[l_2007 + i_2007];
13         }
14
15         for (int j_2007 = 0; j_2007 < n2_2007; ++j_2007) {
16             R_2007[j_2007] = arr_2007[m_2007 + 1 + j_2007];
17         }
18
19         int i_2007 = 0, j_2007 = 0;
20
21         int k_2007 = l_2007;
22
23         while(i_2007 < n1_2007 && j_2007 < n2_2007) {
24             if (L_2007[i_2007] <= R_2007[j_2007]) {
25                 arr_2007[k_2007] = L_2007[i_2007];
26                 i_2007++;
27             } else {
28                 arr_2007[k_2007] = R_2007[j_2007];
29                 j_2007++;
30             }
31             k_2007++;
32         }
33
34         while (i_2007 < n1_2007) {
35             arr_2007[k_2007] = L_2007[i_2007];
36             i_2007++;
37             k_2007++;
38         }
39
40         while (j_2007 < n2_2007) {
41             arr_2007[k_2007] = R_2007[j_2007];
42             j_2007++;
43             k_2007++;
44         }
45     }
46
47     void sort_2007(int arr[], int l_2007, int r_2007) {
48         if (l_2007 < r_2007) {
49
50             int m_2007 = (l_2007 + r_2007) / 2;
51
52             sort_2007(arr, l_2007, m_2007);
53             sort_2007(arr, m_2007 + 1, r_2007);
54
55             Merge_2007(arr, l_2007, m_2007, r_2007);
56         }
57     }
58
59     static void printArray_2007(int arr_2007[]) {
60         int n_2007 = arr_2007.length;
61         for (int i_2007 = 0; i_2007 < n_2007; i_2007++) {
62             System.out.print(arr_2007[i_2007] + " ");
63         }
64         System.out.println();
65     }
66
67     public static void main(String args[]) {
68         int arr_2007[] = {12, 11, 13, 5, 6, 7};
69         System.out.println("Sebelum Terurut");
70         printArray_2007(arr_2007);
71         MergeSort_2411532007 ob_2007 = new MergeSort_2411532007();
72         ob_2007.sort_2007(arr_2007, 0, arr_2007.length - 1);
73         System.out.println("\nSudah Terurut menggunakan merge sort");
74         printArray_2007(arr_2007);
75     }
76
77 }
78
79
80 }
81

```

Penjelasan Operasi

1. Merge_2007() membuat array sementara L_2007 dan R_2007 untuk menyimpan bagian kiri dan kanan.
2. Perulangan while membandingkan elemen kedua bagian dan menempatkan nilai terkecil ke array utama.
3. sort_2007() membagi indeks l_2007 sampai r_2007 menjadi dua bagian dan memanggil dirinya secara rekursif.
4. printArray_2007() menampilkan seluruh isi array sebelum dan sesudah pengurutan.
5. Method main menguji data 12, 11, 13, 5, 6, dan 7.

Hasil Eksekusi

```
sebelum terurut  
12 11 13 5 6 7  
  
Sudah Terurut menggunakan merge sort  
5 6 7 11 12 13
```

Analisis

Array awal belum terurut karena nilai 5, 6, dan 7 berada setelah nilai yang lebih besar. Setelah proses pembagian dan penggabungan, program menghasilkan urutan 5, 6, 7, 11, 12, dan 13. Hasil tersebut menunjukkan bahwa proses rekursif dan method Merge_2007 berhasil menempatkan setiap elemen dalam urutan menaik.

Program QuickSort_2411532007

Class QuickSort_2411532007 menerapkan Quick Sort dengan strategi median of three. Tiga nilai pada posisi awal, tengah, dan akhir dibandingkan untuk membantu menentukan pivot sebelum proses partisi dilakukan.

Source Code QuickSort_2411532007

```

1 package pekan8_2411532007;
2
3 public class QuickSort_2411532007 {
4     static void swap_2007(int[] arr_2007, int i_2007, int j_2007) {
5         int temp_2007 = arr_2007[i_2007];
6         arr_2007[i_2007] = arr_2007[j_2007];
7         arr_2007[j_2007] = temp_2007;
8     }
9
10    static void medianOfThree_2007(int[] arr_2007, int low_2007, int high_2007) {
11        int mid_2007 = low_2007 + (high_2007 - low_2007) / 2;
12
13        if (arr_2007[low_2007] > arr_2007[mid_2007]) {
14            swap_2007(arr_2007, low_2007, mid_2007);
15        }
16        if (arr_2007[low_2007] > arr_2007[high_2007]) {
17            swap_2007(arr_2007, low_2007, high_2007);
18        }
19        if (arr_2007[mid_2007] > arr_2007[high_2007]) {
20            swap_2007(arr_2007, mid_2007, high_2007);
21        }
22        swap_2007(arr_2007, mid_2007, high_2007);
23    }
24
25    static int partition_2007(int[] arr_2007, int low_2007, int high_2007) {
26        medianOfThree_2007(arr_2007, low_2007, high_2007);
27
28        int pivot_2007 = arr_2007[high_2007];
29        int i_2007 = (low_2007 - 1);
30
31        for (int j_2007 = low_2007; j_2007 <= high_2007 - 1; j_2007++) {
32            if (arr_2007[j_2007] < pivot_2007) {
33                i_2007++;
34                swap_2007(arr_2007, i_2007, j_2007);
35            }
36        }
37        swap_2007(arr_2007, i_2007 + 1, high_2007);
38        return (i_2007 + 1);
39    }
40
41    public static void quickSort_2007(int[] arr_2007, int low_2007, int high_2007) {
42        if (low_2007 < high_2007) {
43            int pi_2007 = partition_2007(arr_2007, low_2007, high_2007);
44            quickSort_2007(arr_2007, low_2007, pi_2007 - 1);
45            quickSort_2007(arr_2007, pi_2007 + 1, high_2007);
46        }
47    }
48
49    public static void printArr_2007(int[] arr_2007) {
50        for (int i_2007 = 0; i_2007 < arr_2007.length; i_2007++) {
51            System.out.print(arr_2007[i_2007] + " ");
52        }
53        System.out.println();
54    }
55
56    public static void main (String[] args) {
57        int[] arr_2007 = {10, 7, 8, 0, 1, 5};
58        int N_2007 = arr_2007.length;
59        printArr_2007(arr_2007);
60
61        quickSort_2007(arr_2007, 0, N_2007 - 1);
62
63        System.out.print("Data Terurut quicksort: ");
64        printArr_2007(arr_2007);
65    }
66 }
67
68 }
69
70 }
71

```

Penjelasan Operasi

1. swap_2007() menukar dua elemen pada array.
2. medianOfThree_2007() mengurutkan kandidat awal, tengah, dan akhir lalu menempatkan nilai tengah pada posisi pivot.

3. `partition_2007()` memindahkan nilai yang lebih kecil dari pivot ke sisi kiri dan mengembalikan posisi akhir pivot.
4. `quickSort_2007()` mengurutkan subarray kiri dan kanan secara rekursif.
5. `printArr_2007()` menampilkan isi array sebagai hasil pengujian.

Hasil Eksekusi

```
<terminated> QuickSort_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\java.exe
10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

Analisis

Data awal 10, 7, 8, 9, 1, dan 5 dipartisi berdasarkan pivot. Pemanggilan rekursif dilanjutkan pada bagian kiri dan kanan hingga seluruh subarray memiliki urutan yang benar. Hasil akhir 1, 5, 7, 8, 9, dan 10 membuktikan bahwa partisi dan rekursi berjalan sesuai tujuan.

Program ShellSort_2411532007

Class ShellSort_2411532007 mengurutkan array menggunakan gap yang dimulai dari setengah panjang data. Pada setiap tahap, elemen yang berjarak gap diproses seperti insertion sort. Gap dibagi dua sampai mencapai nol.

Source Code ShellSort_2411532007

```

1 package pekan8_2411532007;
2
3 public class ShellSort_2411532007 {
4     public static void ShellSort_2007(int[] A_2007) {
5         int n_2007 = A_2007.length;
6         int gap_2007 = n_2007/2;
7         while (gap_2007 > 0) {
8             for (int i_2007 = gap_2007; i_2007 < n_2007; i_2007++) {
9                 int temp_2007 = A_2007[i_2007];
10                int j_2007 = i_2007;
11                while (j_2007 >= gap_2007 && A_2007[j_2007 - gap_2007] > temp_2007) {
12                    A_2007[j_2007] = A_2007[j_2007 - gap_2007];
13                    j_2007 = j_2007 - gap_2007;
14                }
15                A_2007[j_2007] = temp_2007;
16            }
17            gap_2007 = gap_2007 / 2;
18        }
19    }
20
21    public static void main (String[] args) {
22        int[] data_2007 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23
24        System.out.print("Sebelum : ");
25        printArray_2007(data_2007);
26
27        ShellSort_2007(data_2007);
28
29        System.out.print("Setelah (Shell Sort) : ");
30        printArray_2007(data_2007);
31    }
32
33    public static void printArray_2007 (int[] arr_2007) {
34        for ( int i_2007 : arr_2007) System.out.print(i_2007 + " ");
35        System.out.println();
36    }
37 }
38
39

```

Penjelasan Operasi

1. Nilai gap_2007 mula-mula ditetapkan sebesar n_2007/2.
2. Elemen pada indeks i_2007 dibandingkan dengan elemen berjarak gap di sebelah kiri.
3. Elemen yang lebih besar digeser sampai posisi yang sesuai ditemukan.
4. Gap diperkecil menjadi setengah dari nilai sebelumnya.
5. printArray_2007() menampilkan data awal dan hasil Shell Sort.

Hasil Eksekusi

```

sebelum : 3 10 4 6 8 9 7 2 1 5
sesudah (Shell Sort) : 1 2 3 4 5 6 7 8 9 10

```

Analisis

Program mengurutkan sepuluh bilangan dari kondisi acak menjadi 1 sampai 10. Perbandingan dengan gap memungkinkan elemen berpindah lebih jauh pada tahap awal. Ketika gap menjadi satu, data telah mendekati kondisi terurut sehingga penyisipan akhir dapat dilakukan secara lebih efisien.

Program BubbleSortGui_2411532007

BubbleSortGui_2411532007 merupakan aplikasi Java Swing untuk memperlihatkan Bubble Sort secara bertahap. Pengguna memasukkan bilangan yang dipisahkan koma, menekan Set Array, lalu menjalankan setiap perbandingan melalui tombol Step.

Source Code BubbleSortGui_2411532007

```

1 package pekan8_2411532007;
2
3 import java.awt.BorderLayout;
4
25 public class BubbleSortGui_2411532007 extends JFrame {
26
27     private JTextField inputField_2007;
28     private JButton setButton_2007;
29     private JButton stepButton_2007;
30     private JButton resetButton_2007;
31     private JTextArea stepArea_2007;
32     private JPanel panelArray_2007;
33
34     private int[] array_2007;
35     private JLabel[] labelArray_2007;
36
37     private int i_2007 = 0;
38     private int j_2007 = 0;
39     private int stepCount_2007 = 1;
40     private boolean sorting_2007 = false;
41
42     public BubbleSortGui_2411532007() {
43         setTitle("Visualisasi Bubble Sort");
44         setSize(800, 500);
45         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46         setLocationRelativeTo(null);
47         setLayout(new BorderLayout(10, 10));
48
49         JPanel inputPanel = new JPanel(new FlowLayout());
50
51         JLabel inputLabel = new JLabel("Masukkan angka (pisahkan dengan koma):");
52         inputField_2007 = new JTextField(30);
53
54         setButton_2007 = new JButton("Set Array");
55         stepButton_2007 = new JButton("Step");
56         resetButton_2007 = new JButton("Reset");
57
58         stepButton_2007.setEnabled(false);
59
60         inputPanel.add(inputLabel);
61         inputPanel.add(inputField_2007);
62         inputPanel.add(setButton_2007);
63         inputPanel.add(stepButton_2007);
64         inputPanel.add(resetButton_2007);
65
66         add(inputPanel, BorderLayout.NORTH);
67
68         panelArray_2007 = new JPanel(new FlowLayout());
69         panelArray_2007.setPreferredSize(new Dimension(750, 120));
70         add(panelArray_2007, BorderLayout.CENTER);
71
72         stepArea_2007 = new JTextArea();
73         stepArea_2007.setEditable(false);
74         stepArea_2007.setFont(new Font("Monospaced", Font.PLAIN, 14));
75
76         JScrollPane scrollPane = new JScrollPane(stepArea_2007);
77         scrollPane.setPreferredSize(new Dimension(750, 250));
78
79         add(scrollPane, BorderLayout.SOUTH);
80
81         setButton_2007.addActionListener(new ActionListener() {
82             @Override
83             public void actionPerformed(ActionEvent e) {
84                 setArrayFromInput();
85             }
86         });
87
88         stepButton_2007.addActionListener(new ActionListener() {
89             @Override
90             public void actionPerformed(ActionEvent e) {
91                 performStep();
92             }
93         });
94
95         resetButton_2007.addActionListener(new ActionListener() {
96             @Override
97             public void actionPerformed(ActionEvent e) {
98                 reset();
99             }
100         });
101     }
102
103     private void setArrayFromInput() {
104         String text = inputField_2007.getText().trim();
105

```

```

105
106     if (text.isEmpty()) {
107         return;
108     }
109
110     String[] parts_2007 = text.split(",");
111     array_2007 = new int[parts_2007.length];
112
113     try {
114         for (int k_2007 = 0; k_2007 < parts_2007.length; k_2007++) {
115             array_2007[k_2007] = Integer.parseInt(parts_2007[k_2007].trim());
116         }
117     } catch (NumberFormatException e) {
118         JOptionPane.showMessageDialog(
119             this,
120             "Masukkan hanya angka yang dipisahkan koma!",
121             "Error",
122             JOptionPane.ERROR_MESSAGE
123         );
124         return;
125     }
126
127     i_2007 = 0;
128     j_2007 = 0;
129     stepCount_2007 = 1;
130     sorting_2007 = true;
131
132     stepButton_2007.setEnabled(true);
133     stepArea_2007.setText("");
134     panelArray_2007.removeAll();
135
136     labelArray_2007 = new JLabel[array_2007.length];
137
138     for (int k_2007 = 0; k_2007 < array_2007.length; k_2007++) {
139         labelArray_2007[k_2007] = new JLabel(String.valueOf(array_2007[k_2007]));
140         labelArray_2007[k_2007].setFont(new Font("Arial", Font.BOLD, 24));
141         labelArray_2007[k_2007].setOpaque(true);
142         labelArray_2007[k_2007].setBackground(Color.WHITE);
143         labelArray_2007[k_2007].setBorder(BorderFactory.createLineBorder(Color.BLACK));
144         labelArray_2007[k_2007].setPreferredSize(new Dimension(50, 50));
145         labelArray_2007[k_2007].setHorizontalAlignment(SwingConstants.CENTER);
146
147         panelArray_2007.add(labelArray_2007[k_2007]);
148     }
149
150     panelArray_2007.revalidate();
151     panelArray_2007.repaint();
152 }
153
154 private void performStep() {
155     if (!sorting_2007 || i_2007 >= array_2007.length - 1) {
156         sorting_2007 = false;
157         stepButton_2007.setEnabled(false);
158         JOptionPane.showMessageDialog(this, "Sorting selesai!");
159         return;
160     }
161
162     resetHighlights();
163
164     StringBuilder stepLog_2007 = new StringBuilder();
165
166     labelArray_2007[j_2007].setBackground(Color.CYAN);
167     labelArray_2007[j_2007 + 1].setBackground(Color.CYAN);
168
169     if (array_2007[j_2007] > array_2007[j_2007 + 1]) {
170         int temp_2007 = array_2007[j_2007];
171         array_2007[j_2007] = array_2007[j_2007 + 1];
172         array_2007[j_2007 + 1] = temp_2007;
173
174         labelArray_2007[j_2007].setBackground(Color.RED);
175         labelArray_2007[j_2007 + 1].setBackground(Color.RED);
176
177         stepLog_2007.append("Langkah ").append(stepCount_2007).append(": ")
178             .append("Menukar elemen ke-").append(j_2007)
179             .append(" (").append(array_2007[j_2007 + 1]).append(")")
180             .append(" dengan ke-").append(j_2007 + 1)
181             .append(" (").append(array_2007[j_2007]).append(")\n");
182     } else {
183         stepLog_2007.append("Langkah ").append(stepCount_2007).append(": ")
184             .append("Tidak ada pertukaran antara ke-")
185             .append(j_2007).append(" dan ke-")
186             .append(j_2007 + 1).append(")\n");
187     }
188 }
189

```

```

189     stepLog_2007.append("Hasil: ").append(arrayToString(array_2007)).append("\n\n");
190     stepArea_2007.append(stepLog_2007.toString());
191
192     updateLabels();
193
194     j_2007++;
195
196     if (j_2007 >= array_2007.length - 1 - i_2007) {
197         j_2007 = 0;
198         i_2007++;
199     }
200
201     stepCount_2007++;
202
203     if (i_2007 >= array_2007.length - 1) {
204         sorting_2007 = false;
205         stepButton_2007.setEnabled(false);
206         resetHighlights();
207
208         for (JLabel label_2007 : labelArray_2007) {
209             label_2007.setBackground(Color.GREEN);
210         }
211
212         JOptionPane.showMessageDialog(this, "Sorting selesai!");
213     }
214 }
215
216 private void updateLabels() {
217     for (int k_2007 = 0; k_2007 < array_2007.length; k_2007++) {
218         labelArray_2007[k_2007].setText(String.valueOf(array_2007[k_2007]));
219     }
220 }
221
222 private void resetHighlights() {
223     for (JLabel label_2007 : labelArray_2007) {
224         label_2007.setBackground(Color.WHITE);
225     }
226 }
227
228 private void reset() {
229     inputField_2007.setText("");
230     panelArray_2007.removeAll();
231     panelArray_2007.remove();
232     panelArray_2007.revalidate();
233     panelArray_2007.repaint();
234
235     stepArea_2007.setText("");
236     stepButton_2007.setEnabled(false);
237
238     sorting_2007 = false;
239     i_2007 = 0;
240     j_2007 = 0;
241     stepCount_2007 = 1;
242 }
243
244 private String arrayToString(int[] arr) {
245     StringBuilder sb_2007 = new StringBuilder();
246
247     for (int k_2007 = 0; k_2007 < arr.length; k_2007++) {
248         sb_2007.append(arr[k_2007]);
249
250         if (k_2007 < arr.length - 1) {
251             sb_2007.append(", ");
252         }
253     }
254
255     return sb_2007.toString();
256 }
257
258 public static void main(String[] args) {
259     SwingUtilities.invokeLater(new Runnable() {
260         @Override
261         public void run() {
262             new BubbleSortGui_2411532007().setVisible(true);
263         }
264     });
265 }
266 }

```

Penjelasan Operasi

1. `setArrayFromInput()` mengubah teks masukan menjadi array integer dan membentuk `JLabel` untuk setiap elemen.
2. `performStep()` membandingkan satu pasangan elemen pada setiap penekanan tombol Step.
3. Warna cyan menandai elemen yang dibandingkan, merah menandai pertukaran, dan hijau menandai proses selesai.
4. `updateLabels()` memperbarui angka pada tampilan setelah perubahan array.

5. reset() menghapus input, visualisasi, log langkah, dan status sorting.

Hasil Eksekusi



Analisis

Pada contoh input 5, 3, 8, 1, dan 4, langkah pertama menukar 5 dengan 3. Langkah kedua tidak melakukan pertukaran antara 5 dan 8, sedangkan langkah ketiga menukar 8 dengan 1. Log di bagian bawah dan perubahan warna label membantu pengguna mengamati proses Bubble Sort satu perbandingan pada satu waktu.

Program MergeSortGui_2411532007

MergeSortGui_2411532007 memvisualisasikan tahap penggabungan pada Merge Sort. Program membentuk daftar rentang merge menggunakan Queue, kemudian setiap penekanan tombol Langkah Selanjutnya menjalankan satu aktivitas, seperti memulai merge, membandingkan elemen, menyalin sisa data, atau menempelkan nilai ke array utama.

Source Code MergeSortGui_2411532007

```

1 package pekan8_2411532007;
2
3 import java.awt.BorderLayout;
4
23 public class MergeSortGui_2411532007 extends JFrame {
24
25     private static final long serialVersionUID_2007 = 1L;
26
27     private int[] array_2007;
28     private JLabel[] labelArray_2007;
29     private JButton stepButton_2007, resetButton_2007, setButton_2007;
30     private JTextField inputField_2007;
31     private JPanel panelArray_2007;
32     private JTextArea stepArea_2007;
33
34     private int i_2007, j_2007, k_2007;
35     private int left_2007, mid_2007, right_2007;
36     private int[] temp_2007;
37
38     private boolean isMerging_2007 = false;
39     private boolean copying_2007 = false;
40     private int stepCount_2007 = 1;
41
42     private Queue<int[]> mergeQueue_2007 = new LinkedList<>();
43
44     public MergeSortGui_2411532007() {
45         setTitle("Merge Sort Langkah per Langkah");
46         setSize(750, 400);
47         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
48         setLocationRelativeTo(null);
49         setLayout(new BorderLayout());
50
51         // Panel input
52         JPanel inputPanel_2007 = new JPanel(new FlowLayout());
53         inputField_2007 = new JTextField(30);
54         setButton_2007 = new JButton("Set Array");
55         inputPanel_2007.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
56         inputPanel_2007.add(inputField_2007);
57         inputPanel_2007.add(setButton_2007);
58
59         // Panel array visual
60         panelArray_2007 = new JPanel();
61         panelArray_2007.setLayout(new FlowLayout());
62         panelArray_2007.setLayout(new FlowLayout());
63
64         // Panel kontrol
65         JPanel controlPanel_2007 = new JPanel();
66         stepButton_2007 = new JButton("Langkah Selanjutnya");
67         resetButton_2007 = new JButton("Reset");
68         stepButton_2007.setEnabled(false);
69         controlPanel_2007.add(stepButton_2007);
70         controlPanel_2007.add(resetButton_2007);
71
72         // Area teks untuk log langkah-langkah
73         stepArea_2007 = new JTextArea(8, 60);
74         stepArea_2007.setEditable(false);
75         stepArea_2007.setFont(new Font("Monospaced", Font.PLAIN, 14));
76         JScrollPane scrollPane_2007 = new JScrollPane(stepArea_2007);
77
78         // Tambahkan panel ke frame
79         add(inputPanel_2007, BorderLayout.NORTH);
80         add(panelArray_2007, BorderLayout.CENTER);
81         add(controlPanel_2007, BorderLayout.SOUTH);
82         add(scrollPane_2007, BorderLayout.EAST);
83
84         // Event Set Array
85         setButton_2007.addActionListener(e_2007 -> setArrayFromInput_2007());
86
87         // Event Langkah Selanjutnya
88         stepButton_2007.addActionListener(e_2007 -> performStep_2007());
89
90         // Event Reset
91         resetButton_2007.addActionListener(e_2007 -> reset_2007());
92     }
93
94     private void setArrayFromInput_2007() {
95         String text_2007 = inputField_2007.getText().trim();
96         if (text_2007.isEmpty()) return;
97
98         String[] parts_2007 = text_2007.split(",");
99         array_2007 = new int[parts_2007.length];
100
101         try {
102             for (int i_2007 = 0; i_2007 < parts_2007.length; i_2007++) {
103                 array_2007[i_2007] = Integer.parseInt(parts_2007[i_2007].trim());
104             }

```



```

104     }
105     } catch (NumberFormatException e_2007) {
106         JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
107             "Error", JOptionPane.ERROR_MESSAGE);
108         return;
109     }
110
111     labelArray_2007 = new JLabel[array_2007.length];
112     panelArray_2007.removeAll();
113
114     for (int i_2007 = 0; i_2007 < array_2007.length; i_2007++) {
115         labelArray_2007[i_2007] = new JLabel(String.valueOf(array_2007[i_2007]));
116         labelArray_2007[i_2007].setFont(new Font("Arial", Font.BOLD, 24));
117         labelArray_2007[i_2007].setOpaque(true);
118         labelArray_2007[i_2007].setBackgroundColor(Color.WHITE);
119         labelArray_2007[i_2007].setBorder(BorderFactory.createLineBorder(Color.BLACK));
120         labelArray_2007[i_2007].setPreferredSize(new Dimension(50, 50));
121         labelArray_2007[i_2007].setHorizontalAlignment(SwingConstants.CENTER);
122         panelArray_2007.add(labelArray_2007[i_2007]);
123     }
124
125     mergeQueue_2007.clear();
126     generateMergeSteps_2007(0, array_2007.length - 1);
127
128     stepButton_2007.setEnabled(true);
129     stepArea_2007.setText("");
130     stepCount_2007 = 1;
131     isMerging_2007 = false;
132     copying_2007 = false;
133
134     panelArray_2007.revalidate();
135     panelArray_2007.repaint();
136 }
137
138 private void generateMergeSteps_2007(int left_2007, int right_2007) {
139     if (left_2007 < right_2007) {
140         int mid_2007 = left_2007 + (right_2007 - left_2007) / 2;
141
142         generateMergeSteps_2007(left_2007, mid_2007);
143         generateMergeSteps_2007(mid_2007 + 1, right_2007);
144
145         mergeQueue_2007.add(new int[] { left_2007, mid_2007, right_2007 });
146     }
147 }
148
149 private void performStep_2007() {
150     resetHighlights_2007();
151
152     if (!isMerging_2007 && !mergeQueue_2007.isEmpty()) {
153         int[] range_2007 = mergeQueue_2007.poll();
154
155         left_2007 = range_2007[0];
156         mid_2007 = range_2007[1];
157         right_2007 = range_2007[2];
158
159         temp_2007 = new int[right_2007 - left_2007 + 1];
160
161         i_2007 = left_2007;
162         j_2007 = mid_2007 + 1;
163         k_2007 = 0;
164
165         copying_2007 = false;
166         isMerging_2007 = true;
167
168         stepArea_2007.append(
169             "Langkah " + stepCount_2007++
170             + ": Mulai merge dari "
171             + left_2007 + " ke "
172             + right_2007 + "\n"
173         );
174
175         return;
176     }
177
178     if (isMerging_2007 && !copying_2007) {
179         if (i_2007 <= mid_2007 && j_2007 <= right_2007) {
180             labelArray_2007[i_2007].setBackgroundColor(Color.CYAN);
181             labelArray_2007[j_2007].setBackgroundColor(Color.CYAN);
182
183             if (array_2007[i_2007] <= array_2007[j_2007]) {
184                 temp_2007[k_2007++] = array_2007[i_2007++];
185             } else {
186                 temp_2007[k_2007++] = array_2007[j_2007++];
187             }
188         }

```

```

188         stepArea_2007.append(
189             "Langkah " + stepCount_2007++
190             + ": Bandingkan dan salin elemen\n"
191         );
192     };
193     return;
194
195     } else if (i_2007 <= mid_2007) {
196         temp_2007[k_2007++] = array_2007[i_2007++];
197
198         stepArea_2007.append(
199             "Langkah " + stepCount_2007++
200             + ": Salin sisa kiri\n"
201         );
202     };
203     return;
204
205     } else if (j_2007 <= right_2007) {
206         temp_2007[k_2007++] = array_2007[j_2007++];
207
208         stepArea_2007.append(
209             "Langkah " + stepCount_2007++
210             + ": Salin sisa kanan\n"
211         );
212     };
213     return;
214
215     } else {
216         copying_2007 = true;
217         k_2007 = 0;
218         return;
219     }
220 }
221
222 if (copying_2007 && k_2007 < temp_2007.length) {
223     array_2007[left_2007 + k_2007] = temp_2007[k_2007];
224     labelArray_2007[left_2007 + k_2007].setText(String.valueOf(temp_2007[k_2007]));
225     labelArray_2007[left_2007 + k_2007].setBackground(Color.GREEN);
226     k_2007++;
227
228     stepArea_2007.append("Langkah " + stepCount_2007++ + ": Tempelkan ke array utama\n");
229     return;
230 }
231
232 if (copying_2007 && k_2007 == temp_2007.length) {
233     isMerging_2007 = false;
234     copying_2007 = false;
235 }
236
237 if (mergeQueue_2007.isEmpty() && !isMerging_2007) {
238     stepArea_2007.append("Selesai.\n");
239     stepButton_2007.setEnabled(false);
240     JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
241 }
242
243 private void resetHighlights_2007() {
244     if (labelArray_2007 == null) return;
245     for (JLabel label_2007 : labelArray_2007) {
246         label_2007.setBackground(Color.WHITE);
247     }
248 }
249
250 private void reset_2007() {
251     inputField_2007.setText("");
252     panelArray_2007.removeAll();
253     panelArray_2007.revalidate();
254     panelArray_2007.repaint();
255     stepArea_2007.setText("");
256     stepButton_2007.setEnabled(false);
257     mergeQueue_2007.clear();
258     isMerging_2007 = false;
259     copying_2007 = false;
260     stepCount_2007 = 1;
261 }
262
263 public static void main(String[] args_2007) {
264     SwingUtilities.invokeLater(() -> {
265         MergeSortGui_2411532007 frame_2007 = new MergeSortGui_2411532007();
266         frame_2007.setVisible(true);
267     });
268 }
269
270 }
271
272 }

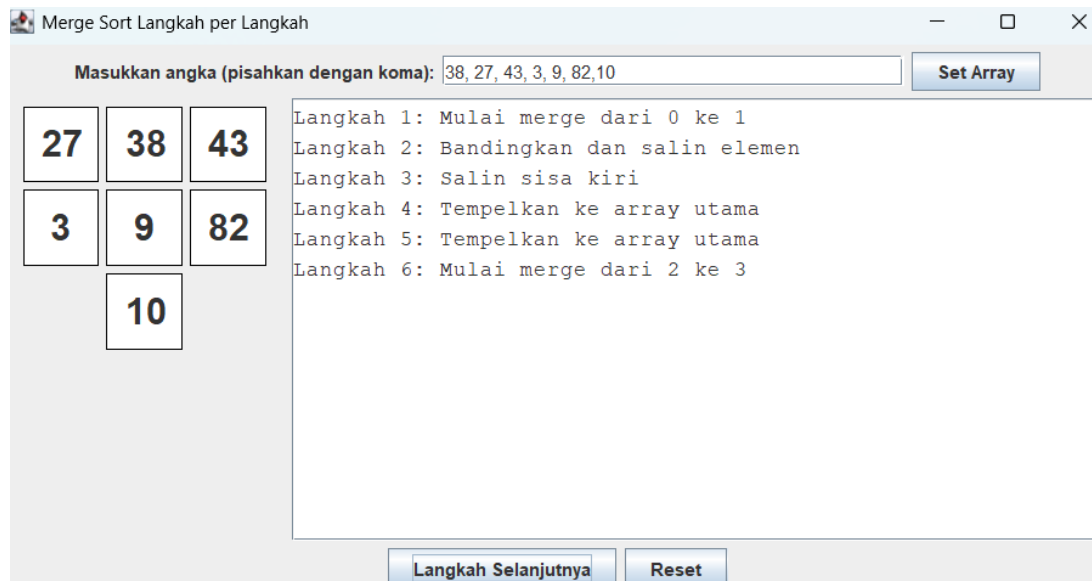
```

Penjelasan Operasi

1. `setArrayFromInput_2007()` memvalidasi input dan membuat komponen label array.
2. `generateMergeSteps_2007()` menyusun urutan rentang yang akan digabungkan secara rekursif ke dalam Queue.

3. `performStep_2007()` mengendalikan proses pembandingan, pengisian array sementara, dan penyalinan kembali.
4. `resetHighlights_2007()` mengembalikan warna label menjadi putih sebelum langkah berikutnya.
5. `reset_2007()` mengosongkan semua data dan status visualisasi.

Hasil Eksekusi



Analisis

Tampilan menunjukkan data 38, 27, 43, 3, 9, 82, dan 10. Setelah beberapa langkah, pasangan pertama telah digabungkan menjadi 27 dan 38, kemudian program mulai memproses rentang berikutnya. Area log menjelaskan aktivitas yang dilakukan sehingga alur pembagian dan penggabungan Merge Sort dapat diikuti secara bertahap.

BAB III

KESIMPULAN

Praktikum 8 berhasil menerapkan Shell Sort, Quick Sort, dan Merge Sort pada array bilangan bulat. Setiap algoritma menggunakan strategi yang berbeda: Shell Sort memakai gap, Quick Sort menggunakan pivot dan partisi, sedangkan Merge Sort membagi dan menggabungkan data secara rekursif.

Visualisasi berbasis Java Swing membantu memperjelas perubahan array pada Bubble Sort dan Merge Sort. Selain itu, penerapan Quick Sort pada objek lagu menunjukkan bahwa algoritma pengurutan dapat digunakan tidak hanya pada bilangan langsung, tetapi juga pada data objek berdasarkan atribut tertentu.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Oracle. (2024). Java Platform, Standard Edition API Specification. Oracle Corporation.

Pradana, N. (2026). Source code Praktikum Struktur Data pekan8_2411532007. GitHub Repository.

LAPORAN TUGAS

Laporan tugas Praktikum 8 menerapkan Quick Sort pada playlist lagu. Class Lagu_2411532007 digunakan sebagai model data, sedangkan Sorting_2411532007 menyimpan kumpulan objek lagu dan mengurutkannya berdasarkan atribut durasi secara ascending.

Tujuan Tugas

1. Membuat class Lagu sebagai representasi judul, penyanyi, dan durasi.
2. Menyimpan beberapa objek Lagu dalam array of object.
3. Mengimplementasikan Quick Sort dengan pemilihan pivot median of three.
4. Mengurutkan playlist berdasarkan durasi lagu secara ascending.
5. Menampilkan data sebelum dan setelah proses sorting.

Class Lagu_2411532007

Lagu_2411532007 merupakan class data yang menyimpan judul, penyanyi, dan durasi. Constructor menginisialisasi seluruh atribut, getter dan setter menyediakan akses terkontrol, sedangkan toString_2007 membentuk tampilan informasi lagu.

Source Code Lagu_2411532007

```

1 package pekan8_2411532007;
2
3 public class Lagu_2411532007 {
4     private String judul_2007;
5     private String penyanyi_2007;
6     private int durasi_2007;
7
8     public Lagu_2411532007(String j_2007, String p_2007, int d_2007) {
9         this.judul_2007 = j_2007;
10        this.penyanyi_2007 = p_2007;
11        this.durasi_2007 = d_2007;
12    }
13
14    public String getJ_2007() {return judul_2007;}
15    public String getP_2007() {return penyanyi_2007;}
16    public int getD_2007() {return durasi_2007;}
17
18    public void setJ_2007(String j_2007) {
19        this.judul_2007 = j_2007;
20    }
21
22    public void setP_2007(String p_2007) {
23        this.penyanyi_2007 = p_2007;
24    }
25
26    public void setD_2007(int d_2007) {
27        this.durasi_2007 = d_2007;
28    }
29
30    public String toString_2007() {
31        return judul_2007 + " - " + penyanyi_2007 + " (" + durasi_2007 + " detik)";
32    }
33
34
35 }
36

```

Penjelasan

1. judul_2007 menyimpan nama lagu.
2. penyanyi_2007 menyimpan nama penyanyi atau musisi.
3. durasi_2007 menyimpan lama lagu dalam satuan detik.
4. Constructor Lagu_2411532007 menerima tiga parameter untuk menginisialisasi objek.
5. Getter dan setter digunakan untuk membaca serta mengubah nilai atribut.
6. toString_2007() menggabungkan judul, penyanyi, dan durasi menjadi satu String.

Class Sorting_2411532007

Sorting_2411532007 mengelola array listLagu_2007 dengan kapasitas dua puluh elemen. Program mengisi tujuh objek lagu, menampilkan playlist, mengurutkan objek berdasarkan durasi menggunakan Quick Sort, lalu menampilkan hasil pengurutan.

Source Code Sorting_2411532007

```

1 package pekan8_2411532007;
2
3
4 public class Sorting_2411532007 {
5     Lagu_2411532007[] listLagu_2007 = new Lagu_2411532007[20];
6     int jumlah_2007 = 0;
7
8     public void inputData_2007() {
9         listLagu_2007[0] = new Lagu_2411532007("Neon Rain", "Neoni", 165);
10        listLagu_2007[1] = new Lagu_2411532007("Take Off", "Quavo & Offset", 195);
11        listLagu_2007[2] = new Lagu_2411532007("Fake It", "Seether", 235);
12        listLagu_2007[3] = new Lagu_2411532007("Anti Hero", "Taylor Swift", 200);
13        listLagu_2007[4] = new Lagu_2411532007("Back To Be Friend", "Noah Cyrus", 210);
14        listLagu_2007[5] = new Lagu_2411532007("Stay With Me", "Sam Smith", 172);
15        listLagu_2007[6] = new Lagu_2411532007("Clarity", "Zedd ft. Foxes", 271);
16        jumlah_2007 = 7;
17    }
18
19    public void tampilData_2007() {
20        for (int i_2007 = 0; i_2007 < jumlah_2007; ++i_2007) {
21            System.out.println((i_2007 + 1) + ". " + listLagu_2007[i_2007].toString_2007());
22        }
23    }
24
25    static void swap_2007(Lagu_2411532007[] arr_2007, int i_2007, int j_2007) {
26        Lagu_2411532007 temp_2007 = arr_2007[i_2007];
27        arr_2007[i_2007] = arr_2007[j_2007];
28        arr_2007[j_2007] = temp_2007;
29    }
30
31    static void medianOfThree_2007(Lagu_2411532007[] arr_2007, int low_2007, int high_2007) {
32        int mid_2007 = low_2007 + (high_2007 - low_2007) / 2;
33
34        if (arr_2007[low_2007].getD_2007() > arr_2007[mid_2007].getD_2007()) {
35            swap_2007(arr_2007, low_2007, mid_2007);
36        }
37        if (arr_2007[low_2007].getD_2007() > arr_2007[high_2007].getD_2007()) {
38            swap_2007(arr_2007, low_2007, high_2007);
39        }
40        if (arr_2007[mid_2007].getD_2007() > arr_2007[high_2007].getD_2007()) {
41            swap_2007(arr_2007, mid_2007, high_2007);
42        }
43        swap_2007(arr_2007, mid_2007, high_2007);
44    }
45
46    static int partition_2007(Lagu_2411532007[] arr_2007, int low_2007, int high_2007) {
47        medianOfThree_2007(arr_2007, low_2007, high_2007);
48
49        int pivot_2007 = arr_2007[high_2007].getD_2007();
50        int i_2007 = (low_2007 - 1);
51
52        for (int j_2007 = low_2007; j_2007 <= high_2007 - 1; j_2007++) {
53            if (arr_2007[j_2007].getD_2007() < pivot_2007) {
54                i_2007++;
55                swap_2007(arr_2007, i_2007, j_2007);
56            }
57        }
58        swap_2007(arr_2007, i_2007 + 1, high_2007);
59        return (i_2007 + 1);
60    }
61
62    static void quickSort_2007(Lagu_2411532007[] arr_2007, int low_2007, int high_2007) {
63        if (low_2007 < high_2007) {
64            int pi_2007 = partition_2007(arr_2007, low_2007, high_2007);
65            quickSort_2007(arr_2007, low_2007, pi_2007 - 1);
66            quickSort_2007(arr_2007, pi_2007 + 1, high_2007);
67        }
68    }
69
70    public void quickSort_2007() {
71        if (jumlah_2007 > 1) {
72            quickSort_2007(listLagu_2007, 0, jumlah_2007 - 1);
73        }
74    }
75
76    public static void main(String[] args) {
77        Sorting_2411532007 playlist_2007 = new Sorting_2411532007();
78
79        System.out.println("=== Sorting Playlist NIM: 2411532007 ===");
80        System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2");
81        playlist_2007.inputData_2007();
82
83        System.out.println("\nData Sebelum Sorting:");
84        playlist_2007.tampilData_2007();

```



```
85  
86     playlist_2007.quickSort_2007();  
87  
88     System.out.println("\nData Setelah Quick Sort (Durasi Asc):");  
89     playlist_2007.tampilData_2007();  
90 }  
91 }
```

Penjelasan

1. inputData_2007() membuat tujuh objek Lagu_2411532007 dan menyimpannya ke dalam array.
2. tampilData_2007() menampilkan objek sampai indeks jumlah_2007.
3. swap_2007() menukar dua referensi objek Lagu pada array.
4. medianOfThree_2007() menentukan kandidat pivot berdasarkan durasi pada posisi awal, tengah, dan akhir.
5. partition_2007() menempatkan lagu dengan durasi lebih pendek di sisi kiri pivot.
6. quickSort_2007() mengurutkan bagian kiri dan kanan secara rekursif.
7. Method main menjalankan pengisian data, penampilan kondisi awal, proses Quick Sort, dan penampilan hasil.

Hasil Eksekusi Laporan Tugas

```
<terminated> Sorting_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\javaw.exe (4 Jul 2026 0
=== Sorting Playlist NIM: 2411532007 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:
1. Neon Rain - Neoni (165 detik)
2. Take Off - Quavo & Offset (195 detik)
3. Fake It - Seether (235 detik)
4. Anti Hero - Taylor Swift (200 detik)
5. Back To Be Friend - Noah Cyrus (210 detik)
6. Stay With Me - Sam Smith (172 detik)
7. Clarity - Zedd ft. Foxes (271 detik)

Data Setelah Quick Sort (Durasi Asc):
1. Neon Rain - Neoni (165 detik)
2. Stay With Me - Sam Smith (172 detik)
3. Take Off - Quavo & Offset (195 detik)
4. Anti Hero - Taylor Swift (200 detik)
5. Back To Be Friend - Noah Cyrus (210 detik)
6. Fake It - Seether (235 detik)
7. Clarity - Zedd ft. Foxes (271 detik)
```

Analisis Hasil

Sebelum diurutkan, lagu tersimpan sesuai urutan input dan durasinya belum ascending. Setelah Quick Sort dijalankan, urutan dimulai dari Neon Rain berdurasi 165 detik, Stay With Me 172 detik, Take Off 195 detik, Anti Hero 200 detik, Back To Be Friend 210 detik, Fake It 235 detik, dan Clarity 271 detik.

Perubahan urutan menunjukkan bahwa pembandingan dilakukan melalui `getD_2007()`. Objek Lagu dipindahkan sebagai satu kesatuan sehingga judul, penyanyi, dan durasi tetap saling terhubung. Pemilihan pivot median of three membantu memilih kandidat pivot dari elemen awal, tengah, dan akhir sebelum partisi.

Kesimpulan Laporan Tugas

Class `Lagu_2411532007` dan `Sorting_2411532007` berhasil membentuk playlist serta mengurutkan objek lagu berdasarkan durasi secara ascending. Implementasi menunjukkan bahwa Quick Sort dapat diterapkan pada array of object dengan menggunakan atribut durasi sebagai kunci pengurutan.

Saran

Program dapat dikembangkan dengan pilihan algoritma interaktif, sorting berdasarkan judul atau penyanyi, input lagu dari pengguna, ArrayList dinamis, dan antarmuka grafis.